EXPERIMENT – 5

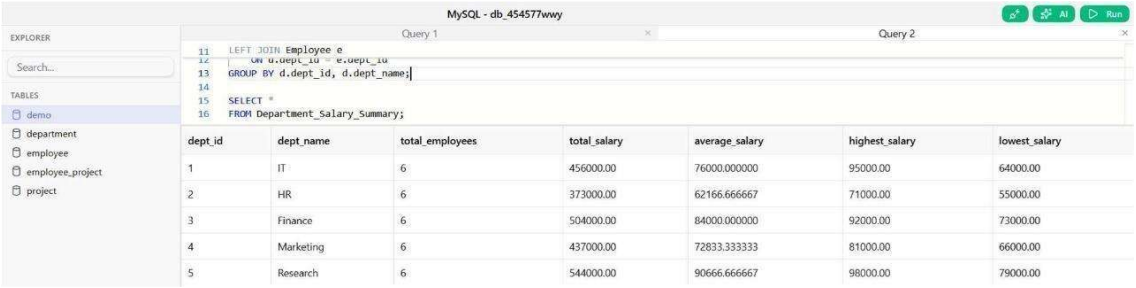
AIM: To create and manipulate SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display employee reporting chains.

SOURCE CODE:
USING EXPERIMENT – 3 DATABASE

SQL QUERIES:
DEPARTMENT SALARY SUMMARY VIEW

```
CREATE VIEW
Department_Salary_Summary AS SELECT
        d.dept_id,
        d.dept_name,
        COUNT(e.emp_id) AS total_employees,
        SUM(e.salary) AS total_salary,
        AVG(e.salary) AS average_salary,
        MAX(e.salary) AS highest_salary,
        MIN(e.salary) AS lowest_salary
FROM Department d LEFT JOIN Employee e
        ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
SELECT *
FROM Department_Salary_Summary;
```

OUTPUT:



dept_id	dept_name	total_employees	total_salary	average_salary	highest_salary	lowest_salary
1	IT	6	456000.00	76000.000000	95000.00	64000.00
2	HR	6	373000.00	62166.666667	71000.00	55000.00
3	Finance	6	504000.00	84000.000000	92000.00	73000.00
4	Marketing	6	437000.00	72833.333333	81000.00	66000.00
5	Research	6	544000.00	90666.666667	98000.00	79000.00

QUERY THE VIEW

```
--Find departments with an average salary greater than ₹75,000 SELECT
dept_name, average_salary
FROM Department_Salary_Summary WHERE average_salary > 75000
```

OUTPUT:



dept_name	average_salary
IT	76000.000000

EMPLOYEE HIERARCHY

```
ALTER TABLE Employee
ADD manager_id INT NULL;
ALTER TABLE Employee
ADD CONSTRAINT fk_employee_manager FOREIGN KEY (manager_id)
REFERENCES Employee(emp_id);
```

INSERT EMPLOYEE HIERARCHY

```
-- Update Manager IDs
UPDATE Employee SET manager_id = NULL WHERE emp_id IN (105, 112, 114, 122,
125);

-- IT
UPDATE Employee
SET manager_id = 105
WHERE emp_id IN (103, 104);
UPDATE Employee
SET manager_id = 103
WHERE emp_id IN (101, 102);
UPDATE Employee
SET manager_id = 104
WHERE emp_id = 106;

-- HR
UPDATE Employee
SET manager_id = 105
WHERE emp_id IN (103, 104)
UPDATE Employee
SET manager_id = 103
WHERE emp_id IN (101, 102);
UPDATE Employee
SET manager_id = 104 WHERE emp_id = 106;

-- Finance UPDATE Employee
SET manager_id = 112
WHERE emp_id IN (107, 108);
UPDATE Employee
SET manager_id = 107 WHERE emp_id = 109;
UPDATE Employee
SET manager_id = 108
WHERE emp_id IN (110, 111)

-- Marketing
UPDATE Employee
SET manager_id = 112
```

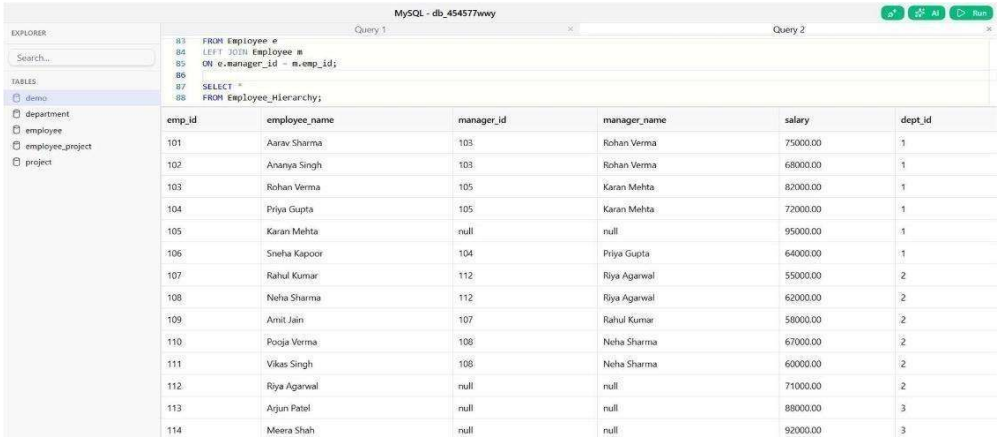
```
UPDATE Employee
SET manager_id = 107 WHERE emp_id = 109;
UPDATE Employee
SET manager_id = 108
WHERE emp_id IN (110, 111);

-- Research UPDATE Employee
SET manager_id = 125
WHERE emp_id IN (126, 127, 128);
UPDATE Employee
SET manager_id = 126
WHERE emp_id = 129;
UPDATE Employee
SET manager_id = 128 WHERE emp_id = 13
```

EMPLOYEE HIERARCHY VIEW

```
CREATE VIEW Employee_Hierarchy AS SELECT
e.emp_id,
e.emp_name AS employee_name, e.manager_id,
m.emp_name AS manager_name, e.salary,
e.dept_id FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;
SELECT *
FROM Employee_Hierarchy;
```

OUTPUT:



emp_id	employee_name	manager_id	manager_name	salary	dept_id
101	Aarav Sharma	103	Rohan Verma	75000.00	1
102	Ananya Singh	103	Rohan Verma	68000.00	1
103	Rohan Verma	105	Karan Mehta	82000.00	1
104	Priya Gupta	105	Karan Mehta	72000.00	1
105	Karan Mehta	null	null	95000.00	1
106	Sneha Kapoor	104	Priya Gupta	64000.00	1
107	Rahul Kumar	112	Riya Agarwal	55000.00	2
108	Neha Sharma	112	Riya Agarwal	62000.00	2
109	Amit Jain	107	Rahul Kumar	58000.00	2
110	Pooja Verma	108	Neha Sharma	67000.00	2
111	Vikas Singh	108	Neha Sharma	60000.00	2
112	Riya Agarwal	null	null	71000.00	2
113	Arjun Patel	null	null	88000.00	3
114	Meera Shah	null	null	92000.00	3

OUTPUT:

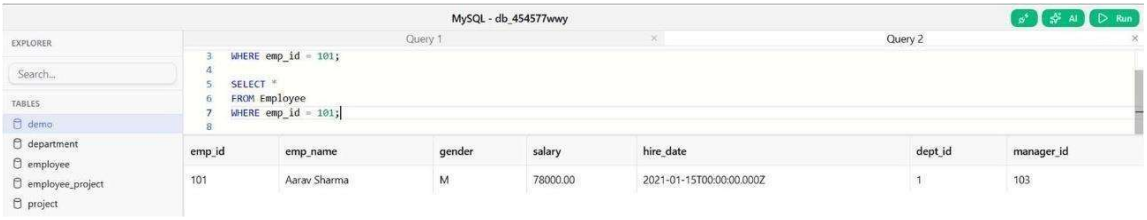


emp_id	emp_name	salary	dept_id
101	Aarav Sharma	75000.00	1

UPDATE THROUGH THE VIEW

```
UPDATE Employee_Basic SET salary = 78000 WHERE emp_id = 101;
SELECT *
FROM Employee WHERE emp_id = 1
```

OUTPUT:



TEST NON-UPDATABILITY OF DEPARTMENT SALARY VIEW

```
UPDATE Department_Salary_Summary
SET average_salary = 80000
WHERE dept_id = 1;
```

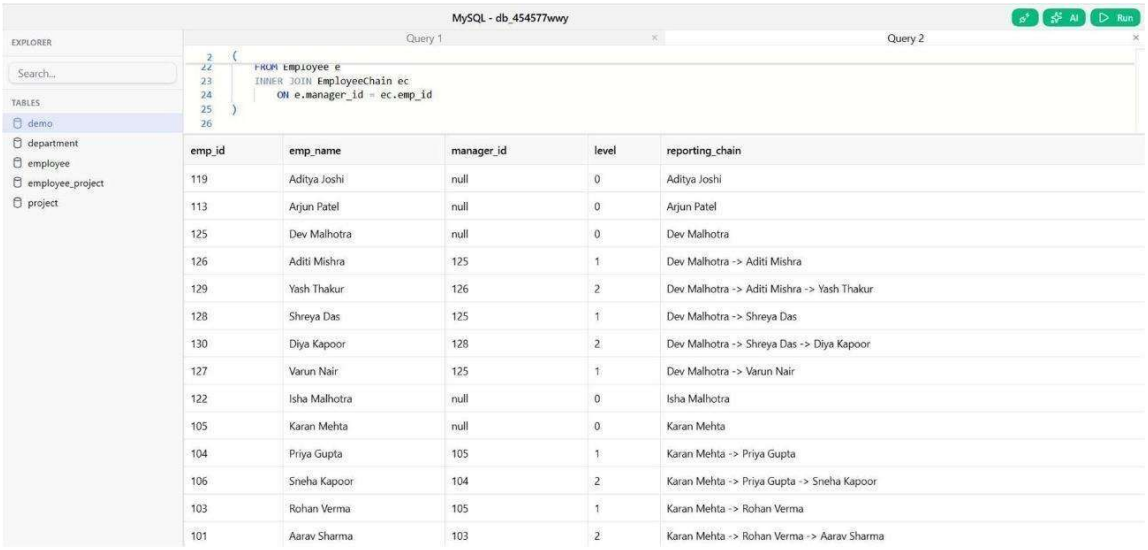
OUTPUT:



RECURSIVE CTE

```
WITH RECURSIVE EmployeeChain AS (
-- Anchor member
SELECT
emp_id, emp_name, manager_id,
0 AS level,
CAST(emp_name AS CHAR(500)) AS reporting_chain FROM Employee
WHERE manager_id IS NULL UNION ALL
-- Recursive member
SELECT
e.emp_id, e.emp_name, e.manager_id, ec.level + 1,
CONCAT(ec.reporting_chain, ' -> ', e.emp_name) FROM Employee e
INNER JOIN EmployeeChain ec
ON e.manager_id = ec.emp_id
)
SELECT
emp_id, emp_name, manager_id, level, reporting_chain
FROM EmployeeChain
ORDER BY reporting_chain;
```

OUTPUT:



emp_id	emp_name	manager_id	level	reporting_chain
119	Aditya Joshi	null	0	Aditya Joshi
113	Arjun Patel	null	0	Arjun Patel
125	Dev Malhotra	null	0	Dev Malhotra
126	Aditi Mishra	125	1	Dev Malhotra -> Aditi Mishra
129	Yash Thakur	126	2	Dev Malhotra -> Aditi Mishra -> Yash Thakur
128	Shreya Das	125	1	Dev Malhotra -> Shreya Das
130	Diya Kapoor	128	2	Dev Malhotra -> Shreya Das -> Diya Kapoor
127	Varun Nair	125	1	Dev Malhotra -> Varun Nair
122	Isha Malhotra	null	0	Isha Malhotra
105	Karan Mehta	null	0	Karan Mehta
104	Priya Gupta	105	1	Karan Mehta -> Priya Gupta
106	Sneha Kapoor	104	2	Karan Mehta -> Priya Gupta -> Sneha Kapoor
103	Rohan Verma	105	1	Karan Mehta -> Rohan Verma
101	Aarav Sharma	103	2	Karan Mehta -> Rohan Verma -> Aarav Sharma

RECURSIVE CTE FOR ONE EMPLOYEE

```
WITH RECURSIVE ReportingChain AS (
SELECT
emp_id, emp_name, manager_id,
0 AS level FROM Employee WHERE emp_id = 101
UNION ALL
SELECT
e.emp_id, e.emp_name, e.manager_id, rc.level + 1
FROM Employee e
JOIN ReportingChain rc
ON e.emp_id = rc.manager_id
)
SELECT
emp_id, emp_name,
manager_id, level
FROM ReportingChain;
```

OUTPUT:

